



Universidad Experimental Nacional de Guayana

Vicerrectorado Académico

Coordinación General de Pregrado

P.F.G: Ingeniería en Informática

Unidad Curricular: INGENIERÍA DE SOFTWARE II

RESOLUCIÓN DE EVALUACIÓN - UNIDAD 3

Estimación en Entornos Complejos de Ingeniería de Software

Profesor:

Marquez Felix

Bachiller:

Alondra Moreno CI: 30.331.870

Samuel Cortez CI: 27.158.842

Carlos Hernández CI: 27.975.753

Eduardo Quintero CI: 32.004.965

Ciudad Guayana, Septiembre 2026

1. RESOLUCIÓN DEL EJERCICIO 1: Métodos de Estimación

Parte A: Estimación Predictiva (PERT)

Datos del problema: O = 10, M = 20, P = 60 días-persona.

Cálculo paso a paso:

La fórmula de estimación de esfuerzo esperado (Te) de la distribución Beta (PERT) es:

$$Te = (O + 4M + P) / 6$$

$$Te = (10 + 4*20 + 60) / 6$$

$$Te = (10 + 80 + 60) / 6$$

$$Te = 150 / 6$$

$$Te = 25 \text{ días-persona.}$$

Análisis Analítico y Conceptual (Asimetría del Riesgo):

La media ponderada (Te = 25) resulta considerablemente mayor que la estimación más probable (M = 20) debido al fenómeno conocido como la 'asimetría del riesgo' en el desarrollo de software. Si observamos los datos, el escenario optimista (10) está a solo 10 días de distancia del escenario más probable, pero el escenario pesimista (60) se aleja por 40 días.

En la ingeniería de software, las tareas casi nunca se completan mucho más rápido de lo esperado, pero cuando ocurren problemas (deuda técnica, bugs complejos, caídas de servidores), los retrasos pueden ser masivos.

La fórmula PERT captura esta 'cola pesada' estadística, arrastrando el valor esperado hacia la derecha para proteger el cronograma contra la incertidumbre inherente del código.

Parte B: Métricas de Flujo y Giro Ágil

Datos del problema: Backlog = 90 Story Points (SP), Throughput Promedio = 30 SP/Sprint, Desviación Estándar = 5.

En agilidad, no intentamos adivinar fechas exactas, sino que proyectamos escenarios probabilísticos basados en datos reales (empirismo).

- Cálculo Base: Tiempo (Sprints) = Backlog / Throughput = 90 / 30 = 3 Sprints.
- Aplicación de Monte Carlo: Como existe una desviación estándar (5), sabemos que el equipo no entrega 30 SP exactos siempre. Podrían entregar 25, 35 o incluso 20. La simulación de Monte Carlo ejecuta miles de escenarios matemáticos simulados con este rango de variabilidad para calcular una curva de probabilidad. En lugar de decir 'Entregaremos en el Sprint 3', la simulación nos permite afirmar: 'Tenemos un 85% de confianza en que entregaremos entre el Sprint 3 y el Sprint 4'.

Por qué este enfoque empírico es más confiable que PERT:

La estimación determinista como PERT asume que podemos predecir el futuro adivinando variables (O, M, P) basadas en juicios humanos subjetivos (que sufren de optimismo injustificado). Por el contrario, las métricas de flujo y Monte Carlo se basan en la realidad histórica del equipo (Throughput). En entornos complejos (como el desarrollo de software), el empirismo (basado en lo que ya ocurrió) siempre será más preciso y realista que el determinismo (basado en suposiciones).

2. RESOLUCIÓN DEL EJERCICIO 2: Sobrecarga y Ley de Brooks

Caso: El Director ordena duplicar el equipo (de 5 a 10 programadores) a 2 meses de la fecha límite para recuperar tiempo.

Parte A: Análisis Matemático y Conceptual

La fórmula industrial tradicional (Tiempo = Esfuerzo / Recursos) asume erróneamente que las tareas de software son perfectamente divisibles, como poner ladrillos. Fred Brooks, en su célebre obra 'El Mítico Hombre-Mes', sentenció:

'Añadir recursos humanos a un proyecto de software retrasado lo retrasará aún más' (Ley de Brooks). Esto ocurre porque el desarrollo de software es un trabajo intelectual altamente interdependiente.

Sumar personas en etapas tardías incrementa exponencialmente el caos, especialmente si consideramos el 'Cono de Incertidumbre', ya que la visión inicial del proyecto difiere drásticamente de la realidad técnica de las últimas fases.

Parte B: Los 3 Factores de Sobrecarga

- **1. Sobrecarga de Comunicación (Communication Overhead):** La fórmula de canales de comunicación es $C = n(n-1)/2$.
Con el equipo original ($n=5$): $C = 5(4)/2 = 10$ canales de comunicación.
Al duplicar el equipo ($n=10$): $C = 10(9)/2 = 45$ canales.

Conclusión: Incrementar el equipo un 100% genera un aumento del 350% en la complejidad comunicacional.

El equipo perderá innumerables horas alineando ideas, sincronizando avances y asistiendo a reuniones en lugar de programar.

- **2. Onboarding y Curva de Aprendizaje:** Los 5 nuevos desarrolladores no conocen la arquitectura del software, las reglas de negocio ni el entorno de desarrollo. Para integrarlos, los 5 programadores originales deberán detener su trabajo para explicar el código, supervisar y corregir a los novatos. Esto produce una caída en picada de la productividad general a corto plazo.
- **3. Integración y Fricción Técnica:** Más manos sobre el mismo teclado generan colisiones. Al tener 10 personas escribiendo código simultáneamente en los mismos repositorios, aumentarán exponencialmente los conflictos de fusión (merge conflicts) en Git. La falta de contexto de los nuevos introducirá bugs críticos, aumentando la 'deuda técnica' justo a 2 meses de la entrega.

Parte C: Alternativas de Gestión Estables

- **1. Enfoque Predictivo Tradicional:** Aplicar 'Desalcance' (Scope Reduction) para eliminar requerimientos secundarios, o aplicar 'Fast-Tracking' (paralelizar tareas).

Si se debe añadir personal, solo hacerlo en tareas de la Ruta Crítica (CPM) que sean completamente independientes (como QA o testing manual), no en desarrollo core.

- **2. Enfoque Ágil / Híbrido:** Congelar el ingreso de nuevo personal y re-priorizar el Backlog junto al Product Owner. Aplicar la regla de Pareto (80/20) para definir un Producto Mínimo Viable (MVP) estricto.

Negociar con el cliente para entregar el core del sistema en la fecha acordada y trasladar las funcionalidades menos críticas a sprints posteriores.

3. GUÍA PARA EL EJERCICIO 3: WBS y Planificación de AuraPulse (Project Manager)

A continuación, se detalla la estructura jerárquica de tareas (EDT/WBS) para el proyecto AuraPulse de telemedicina IoT, formateada explícitamente para ser copiada y pegada en MS Project o ProjectLibre. Esta estructura incluye duraciones realistas y dependencias lógicas obligatorias.

Estructura EDT/WBS para MS Project:

#	Fila	Nombre (Name)	Duración (Duration)	# Predecesoras
	1	1.0 Fase de Planificación y Arquitectura	2w	
	2	1.1 Project Charter y Análisis	1w	
	3	1.2 Diseño de Arquitectura Cloud	1w	2
	4	2.0 Fase de Diseño UX/UI	2w	
	5	2.1 Wireframes y Journey	1w	2
	6	2.2 Prototipo Alta Fidelidad	1w	5
	7	3.0 Desarrollo de Backend e Integración	3w	
	8	3.1 Construcción Base de Datos	1w	3
	9	3.2 Desarrollo API REST	2w	8
	10	4.0 Motor de Reglas Clínicas	3w	
	11	4.1 Algoritmos de Detección	2w	9
	12	4.2 Integración Motor de Alertas	1w	11
	13	5.0 Desarrollo Frontend	3w	
	14	5.1 Portal de Autenticación	1w	6
	15	5.2 Construcción del Dashboard	2w	9;14
	16	6.0 Pruebas y Aseguramiento QA	2w	
	17	6.1 Pruebas de Estrés	1w	12;15
	18	6.2 Auditoría de Seguridad	1w	17
	19	7.0 Despliegue a Producción	1w	
	20	7.1 Configuración Servidores	3d	18
	21	7.2 Go-Live y Monitoreo	2d	20

Al ingresar estas tareas secuencialmente, el software generará automáticamente:

- El Diagrama de Gantt ubicando las 16 semanas (si se suman las superposiciones correctas o dependencias directas en cascada).
- El Diagrama de Red (PERT/CPM) trazará en rojo la Ruta Crítica, que en este caso atravesará el Backend, el Motor de Reglas Clínicas y las Pruebas QA, identificando exactamente dónde NO pueden permitirse retrasos.

